Unidad 2.
Manejo de conectores

2.1. Introducción.
2.2. El desfase objeto-relacional.
2.3. Bases de datos embebidas.
2.4. Protocoles de acceso a bases de datos.
2.5. Acceso a datos mediante ODBC.
2.6. Acceso a datos mediante JDBC
2.7. Establecimiento de conexiones.
2.8. Ejecución de sentencias de descripción de datos.
2.9. Ejecución de sentencias de manipulación de datos.
2.10 Ejecución de procedimientos.
2.11. Gestión de errores.
2.12. Patrón modelo-vista-controlador. Acceso a datos.

2.1 Introducción

- Acceso a datos: recuperación o modificación de datos locales o remotos.
- El origen de los datos no es sólo relacional; puede ser una hoja de cálculo, un fichero de texto, una base de datos relacional, un servicio de información on line, etc.
- Los conectores son el software necesario para realizar conexiones desde un programa Java con una base de datos relacional.

2.2 El desafío objeto-relacional

- Las BDOO solucionan necesidades a las que no pueden dar respuesta las BDR: aplicaciones de ingeniería, GIS, experimentos científicos, multimedia, etc.
- Los objetos son elementos complejos sobre los que se fundamenta la POO. Las BDR no son capaces de almacenar este tipo de elementos.
- El almacenamiento de objetos es más complejo que el de las tablas relacionales; esto origina el desfase objeto-relacional.
- Cada vez que los objetos deben almacenarse/extraerse en una BDR se requiere un mapeo desde las estructuras del modelo relacional a las del entorno de programación.

2.3 Bases de datos embebidas

Una BD embebida se utiliza cuando esta no contiene gran cantidad de datos. El motor de estas bases de datos queda incrustado en la aplicación y es de uso exclusivo en dicha aplicación.

2.3.1. SQLITE

- Es un SGBD escrito en C donde las BD se guardan en forma de ficheros.
- La biblioteca implementa la mayor parte del estándar SQL-92: transacciones, triggers, consistencia y la mayoría de consultas complejas.
- Instalación. Desde la página **http://www.sqlite.org/download.html** descargar el fichero zip **sqlite-shell-win32-x86-3070900.zip** y ejecutar el fichero **sqlite3.exe.**
- Al ejecutarlo desde la línea de comandos escribiremos el nombre de la BD; si no existe, se creará.

2.3 Bases de datos embebidas

Ejemplo SQLITE

```
D:\>sqlite3 d:\ejemplo.db
SQLite version 3.7.14 2012-09-03 15:42:36
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> BEGIN TRANSACTION;
sqlite> CREATE TABLE departamentos (
   ...> dept_no TINYINT(2) NOT NULL PRIMARY KEY,
   ...> dnombre VARCHAR(15),
   ...> loc VARCHAR(15)
   ...> );
sqlite>
sqlite> INSERT INTO departamentos VALUES (10, 'CONTABILIDAD', 'SEVILLA');
sqlite> INSERT INTO departamentos VALUES (20, 'INVESTIGACIÓN', 'MADRID');
sqlite> INSERT INTO departamentos VALUES (30, 'VENTAS', 'BARCELONA');
sqlite> INSERT INTO departamentos VALUES (40, 'PRODUCCIÓN', 'BILBAO');
sqlite> COMMIT;
sqlite> .quit
```

2.3 Bases de datos embebidas

2.3.2. APACHE DERBY

- Es una base de datos relacional de código abierto implementada en Java.
- Soporta estándares SQL y ofrece un controlador integrado JDBC que permite incrustar Derby en cualquier solución basada en Java.
- Soporta el paradigma cliente servidor.
- Instalación. Desde la página. **http://db.apache.org/derby/derby_downloads.html** descargamos el fichero **dbderby-10.8.2.2-bin.zip** lo descomprimimos, por ejemplo, en la carpeta **D:\db-derby-10-8-2-2-bin.**
- Para utilizar derby en los programas Java será necesario tener accesible la librería derby.jar en el CLASSPATH de nuestro programa.
- Apache Derby incluye un fichero IJ.BAT que permite ejecutar por consola órdenes para crear nuestras BD y ejecutar sentencias DDL y DML.

D:\db-derby-10.8.2.2-bin\bin>ij

- Para crear una BD de nombre ejemplo en la carpeta D:\db\Derby escribimos, desde el indicador **ij>** lo siguiente:

connect ‘jdbc:derby:D\db\erby\ejemplo;create=true’;

2.3 Bases de datos embebidas

2.3.2. APACHE DERBY

connect ‘jdbc:derby:D\db\erby\ejemplo;create=true’;

- **Connect** es el comando para establecer la conexión.
- **jdbc:derby** es el protocolo JDBC especificado por DERBY.
- **ejemplo** es el nombre de la base de datos que voy a crear.
- **create=true** es el atributo usado para crear a base de datos.

Para salir de la línea de comandos de IJ escribimos **exit.**

2.3 Bases de datos embebidas

2.3.3. HSQLDB

- Es un SGBD relacional escrito en Java. La suite ofimática OpenOffice ver 2.0 lo incluye para dar soporte a la aplicación Base.
- Permite integridad referencial, procedimientos almacenados en Java, disparadores y tablas de hasta 8Gb.
- Descargamos desde http://sourceforge.net/projects/hsqldb/files descargamos la versión para Windows **hsqldb-2.2.6.zip**. Al descomprimirlo obtendremos el fichero **hsqldb** que copiaremos en la carpeta **D:\hsqldb**.
- Creamos la carpeta **D:\hsqldb\data** para guardar ahí los ficheros de BD que creemos.
- Desde **D:\hsqldb\bin** ejecutamos el archivo **runUtil.BAT** con el parámetro DatabaseManager para conectarnos a la BD y que se ejecute la interfaz gráfica del SGBD.

2.3 Bases de datos embebidas

2.3.4. H2 (YA no esta disponible)

- Es un SGBD relacional escrito en Java.
- Descargamos desde http://sourceforge.net/projects/hsqldb/files descargamos la versión para Windows **h2-2011-11-26.zip**.
- Lo descomprimimos en D:/h2 y en la carpeta D:\h2ºbin ejecutamos el fichero h2.bat.
- Se abre el navegador (fig 2.3 pág. 56). Escribimos un nombre para la configuración de la BD.
- Escribimos la URL para la conexión a nuestra BD: jdbc:h2:D/db/H2/ejemplo/ejemplo.
- Guardar la configuración y Conectar para establecer la conexión a la BD.

2.3 Bases de datos embebidas

2.3.5 DBO4

Es un motor de bases de datos orientado a objetos, disponible para entornos Java y .NET. Características:

- Se evita el problema del desfase objeto-relacional.
- No existe un lenguaje SQL para la manipulación de datos, sino métodos delegados.
- Se instala añadiendo un único fichero de librería (JAR para Java o DLL para .NET).
- Se crea un único fichero de BA con la extensión .YAP.
- Se descarga desde http://www.db4o.com el archivo **db4o-8.0-java.zip**. Al descomprimirlo hay una carpeta *lib* donde se encuentran los JAR (db4o.jar) que hay que agregar a nuestro proyecto para utilizar el motor de BD.

2.4. Protocolos de acceso a datos mediante ODBC

Existen dos normas de conexión a una BD SQL:

- ODBC. (*Open Data Base Connectivity*). Define un API que pueden usar las aplicaciones para abrir uan conexión con una BD, enviar consultas o actualizaciones.
- JDBC (*Java Data Base Conectivity*). Define un API que pueden usar los programas Java para conectarse a los servidores de BD relacionales.

OLE_DB Es un API para datos que no son bases de datos.

ADO es un API que ofrece una interfaz sencilla de utilizar por la funcionalidad OLE-DB.

2.5. Acceso a datos mediante ODBC

- ODBC: Estándar de acceso a datos de Microsoft para posibilitar el acceso a datos desde cualquier aplicación con independencia del gestor de BD.
- Cada SGBD compatible con ODBC debe proporcionar una biblioteca que se debe enlazar con el programa cliente.
- Cuando el programa cliente hace una llamada a la API ODBC el código de la biblioteca se comunica con el servidor para realzar la acción solicitada y obtener los resultados.

Pasos para usar ODBC

1. Configurar la interfaz ODBC, asignando un entorno SQL con la función *SQLAllocHandle()* y un manejador para la conexión a la BD en el entorno anterior. Así, el manejador traducirá las consultas de datos de la aplicación en comandos que el SGBD entienda. ODBC define los siguientes manejadores:
   - SQLHENV. Define el entorno de acceso a datos.
   - SQLHDBC. Identifica el estado y configuración de la conexión.
   - SQLHSTMT. Declaración SQL y conjunto de resultados asociados.
   - SQLHDESC. Recolección de metadatos usados para describir sentencias SQL.

2.5. Acceso a datos mediante ODBC

Pasos para usar ODBC (continuación)

2. Una vez reservados los manejadores, el programa abre la conexión a la BD usando *SQLDriverConnect()* o *SQLConnect().*

3. Una vez realzada la conexión el programa puede enviar órdenes SQL a la BD utilizando SQLExecDirect().

4. Al finalizar la sesión el programa se desconecta de la BD y libera la conexión y los manejadores del entorno SQL.

La API ODBC tiene una interfaz escrita en C y no es apropiada para su uso directo desde Java; existen inconvenientes de seguridad, implementación, robustez y portabilidad de las aplicaciones.

2.6. Acceso a datos mediante JDBC

- JDBC proporciona una librería estándar para acceder a fuentes de datos principalmente orientados a BDR que usan SQL.
- JDBC dispone de una interfaz distinta para cada BD; un driver o conector, lo que permite que las llamadas a los métodos Java de las clases JDBC se correspondan con el API de la BD.

JDBC consta de un conjunto de clases e interfaces que nos permiten escribir aplicaciones Java para gestionar las siguientes tareas con una BDR:

- Conectarse a la BD.
- Enviar consultas e instrucciones de actualización a la BD.
- Recuperar y procesar los resultados recibidos de la BD en respuesta a las consultas.

2.6. Acceso a datos mediante JDBC

2.6.1. Arquitecturas JDBC

La API JDBC es compatible con los modelos de dos y tres capas para el acceso a BD.

- Modelo de 2 capas. Desde el programa Java se envían sentencias SQL al SGBD, que las procesa y devuelve al programa. El driver se encarga de manejar la comunicación a través de la red de forma transparente al programa.

- Modelo de 3 capas. Los comandos se envía a una capa intermedia que se encarga de enviar los comandos SQL a la BD y de recoger los resultados de la ejecución de las sentencias.

2.6. Acceso a datos mediante JDBC

2.6.2. Tipos de drivers

- **Tipo 1. JDBC-ODBC Bridge**. Permite el acceso a BD JDBC mediante un driver ODBC. Exige la instalación de ODBC en la máquina cliente.
- **Tipo 2. Native.** Traduce las llamadas al API de JDBC Java en llamadas propias de motor de BD. Exige instalar en la máquina cliente código binario propio del cliente de BD y del sistema operativo.
- **Tipo 3. Network.** Controlador de Java puro que utiliza un protocolo de red para comunicarse con un servidor de BD. Traduce las llamadas al API de JDBC Java en llamadas propias del protocolo de red usado por el motor de BD. No exige instalación en cliente.
- **Tipo 4. Thin.** Controlador de Java puro: Traduce las llamadas al API de JDBC Java en llamadas propias del protocolo de red usado por el motor de BD. No exige instalación en cliente.

2.6.3. Como funciona JDBC

DBC define varias interfaces que permiten realizar operaciones copn BD. A partir de ellas se derivan las clases correspondientes, definidas en el paquete **java.sql** (tabla pag.71).

2.6. Acceso a datos mediante JDBC

2.6.3. Como funciona JDBC

Ejemplo que muestra las 4 clases principales que usa un programa Java con JDBC.

```
import java.sql.*;1
public class Main {
  public static void main(String[] args {
  try {
   Class.forName("com.mysql.jdbc.Driver");2
   Connection conexión = DriverManager.getConnection 4
   ("jdbc:mysql://localhost/ejemplo", "ejemplo", "ejemplo");3
   Statement sentencia = conexión. createStatement();5
   Result result = sentencia.executeQuery("SELECT * FROM
   departamentos");6
   while (result.next())7
     {system.out.println(result.getInt(1) + " " + result.getString(2)
     + " " result.getString(3));}
   result.close();8
   sentencia.close();9
   conexión.close();10
   }
   catch (ClassNotFoundException cn) {cn.printStackTrace();}
   catch (SQLException e) {e.printStackTrace();}
}}
```

2.6. Acceso a datos mediante JDBC

2.6.4. Acceso a datos mediante el puente JDBC-ODBC

- Para aquellos productos donde no hay controlador JDBC y si uno ODBC existe un puente JDBC-ODBC. Se trata de un driver que implementa operaciones JDBC traduciéndolas en operaciones ODBC.
- El puente se instala automáticamente con el JDK como el paquete *sun.jdbc.odbc*.
- Ejemplo: Para acceder a una BD MySQL usando el puente JDBC-ODBC hay que crear un origen de datos on DNS (Data Source Name). En Windows sería:

- ✓ *Panel de Control-> Herramientas Administrativasa -> Origen de datos (ODBC)*
- ✓ Una vez ahí, *Agregar* y seleccionar el driver ODBC para MySQL y *Finalizar*.
- ✓ Damos nombre al origen de los datos; por ejemplo *Mysql-odbc*.
- ✓ Escribimos el nombre de la máquina (dirección IP) donde se encuentra ese origen. Si es la misma máquina, ponemos *localhost*.
- ✓ Como nombre de usuario ponemos uno que exissta e la BD, por ejemplo, *ejemplo y su password.*
- ✓ De la lista DATABASE elegimos un esquema de BD, en este caso EJEMPLO y OK.

2.6. Acceso a datos mediante JDBC

2.6.4. Acceso a datos mediante el puente JDBC-ODBC

- Por cada esquema de BD es necesario crear un origen de datos. Desde *Connect Options* podemos escribir el número de puerto por el que escucha MySQL; por defecto 3306.
- A continuación, se visualiza la pantalla inicial del *Administrador de Orígenes de datos ODBC* con el nuevo origen creado y pulsamos *Aceptar*.
- Si el driver ODBC para conectar MySQL no está instalado, lo descargamos:

  http://www.mysql.com/products/connectors y descargamos el driver:

  ODBC Driver for MySQL (Connector/ODBC)

  Ejecutamos el fichero *mysql-connector-odbc-5.1.9-win32.msi*
- En el programa Java que estamos siguiendo cambiamos dos líneas

```
// Cargar el driver
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
//Establecemos la conexión  con la BD
Connection conexión =
DriveManager.getConnection("jdbc:odbc:Mysql-odbc");
```

2.7 Establecimiento de conexiones

- Vamos a conectarnos a través de JDBC a las bases de datos embebidas vistas anteriormente. Para ello hay que tener creada la BD EJEMPLO con las tablas EMPLEADOS y DEPARTAMENTOS. Utilizaremos el mismo programa Java y sólo cambiaremos la carga del driver y la conexión a la BD.

  - ✓ Conexión a H2. Librería *h2-1-3-162.jar* y fichero *h2-2011-11-26.zip*.
  - ✓ Conexión a MySQL. Librería *mysql-connector-java-5-1-18-bin.jar*, y fichero *com.mysql.jdbc.Driver.*
  - ✓ Conexión a Oracle.
  - ✓ Conexión a Access. Elegimos como origen de los datos *Microsoft Access Driver (*.mdb, *.accdb)* o *Microsoft Access Driver (*.mdb).*

2.8 Ejecución de sentencias de descripción de datos

- Cuando desarrollamos una aplicación JDBC conocemos la estructura de las tablas, los datos que estamos manejando y las relaciones que hay.
- Si desconocemos la estructura de la BD, puede obtenerse a través de los *metaobjetos*.
- El objeto **DatabaseMetaData** proporciona información sobre la BD a partir de los métodos que contiene.

Ejemplo (pág. 83). Conectar con la BD MySQL de nombre *ejemplo* y mostrar información sobre el producto, el driver, la URL, el nombre de usuario y las tablas y vistas del esquema actual

2.8 Ejecución de sentencias de descripción de datos

2.8.1. RESULTSETMETADATA

Se trata de una interfaz que permite obtener metadatos a partir de un *ResultSet*. Significa que podemos obtener información sobre los tipos y propiedades de las columnas, como el número de columnas devuelto. Su principales métodos son:

- *getColumnCount().* Devuelve el número de columnas devueltas por una consulta.
- *getColumnName(índice de la columna).* Devuelve el nombre de la columna.
- *getColumnTypeName(índice).* Devuelve el nombre del tipo de dato que contiene la columna específico del sistema de bases de datos.
- *isNullable(índice).* Devuelve 0 si la columna puede contener valores nulos.
- *getColumnDisplaySize(índice).* Devuelve el máximo ancho en caracteres de la columna.

Ejemplo pág. 87

2.9. Ejecución de sentencias de manipulación de datos

La interfaz **Statement** dispone de métodos que permiten ejecutar sentencias SQL.
Al tratarse de una interfaz, no puede crear objetos directamente; estos se crean con el método *createStatement()* de un objeto *Connection* válido.

Statement sentencia = conexión.createStatement()

Al crear un objeto **Statement** se crea un espacio de trabajo para crear consultas SQL, ejecutarlas y obtener sus resultados. Los métodos disponibles son:

- ✓ *executeQuery(String).* Para sentencias SQL que recuperan datos (SELECT) de un único objeto *ResultSet*
- ✓ *executeUpdate(String).* Se utiliza con sentencias de manipulación de datos (DML): INSERT, UPDATE, DELETE y de definición de datos (DDL): CREATE, DRPO, ALTER.
- ✓ *Execute(String).* Se utiliza para sentencias que devuelven más de un *ResultSet*, como por ejemplo la ejecución de procedimientos almacenados.

Ejemplos pág. 89 y 90

2.9. Ejecución de sentencias de manipulación de datos

2.9.1. Sentencias preparadas

- La interfaz **PreparedStatement** nos va a permitir construir una cadena de caracteres SQL con *placeholder* (marcadores de posición) que representa los datos que serán asignados más tarde. El *placeholder* se representa con el símbolo **?**.

Ej.: **INSERT INTO departamentos VALUES (15, 'INFORMÁTICA', 'MADRID')**

```
String sql = "INSERTO INTO departamentos VALUES (?, ?, ?);
// 1 2 3  valores de índice.
```

- Antes de ejecutar un **PreparedStatement** es necesario asignar los datos para que cuando se ejecute la BD asigne variables de unión con estos datos y ejecute la orden SQL.

- Los objetos **PreparedStatement** se pueden precompilar una sola vez y ejecutarse las que queramos asignando diferentes valores a los marcadores de posición. Por el contrario, en los objetos **Statement** la sentencia SQL se suministra en el momento de ejecutar la sentencia.

2.9. Ejecución de sentencias de manipulación de datos

2.9.1. Sentencias preparadas

- Los métodos de **PreparedStatement** tienen los mismos nombres *(executeQuery(), executeUpdate() y execute())* que en **Statement** pero no se necesita enviar la cadena de caracteres con la orden SQL en la llamada ya que lo hace el método *prepareStatement(String).*

```
PreparedStatement sentencia = conexión.prepareStatement(sql);
```

- Se utilizan los métodos *setInt(índice, entero)*, *setString(índice, cadena)* y *setFloat(índice, float)* para asignar los valores a cada uno de los marcadores de posición. Con *Integer.parseInt(dep)* convertimos la cadena *dep* a entero y con *Float.parseFloat(subida*) convertimos la cadena *subida* a float.

- Se puede utilizar esa interfaz con la orden SELECT. (Ejemplo pág.92)

2.10. Ejecución de procedimientos

- Los procedimientos almacenados en la BD consiste en un conjunto de sentencias SQL y del lenguaje procedural que utiliza el SGBD para las tareas sobre la BD.
- Pueden definirse con parámetros de entrada (IN), salida (OUT), entrada/salida (IN/OUT) o sin parámetros.

```
CREATE PROCEDURE subida_sal (d INT, subida INT)
BEGIN
 UPDATE empelados SET salario = salario + subida WHERE dept_no=d;
 COMMIT;
END;
```

```
CREATE OR REPLACE FUNCTION nombre_dep (d NUMBER, locali OUT varchar2)
          return varchar2 as nom VARCHAR2(15);
BEGIN
  SELECT dnombre, loc, INTO nom, locali FROM departamentos WHERE
          dept_no=d; return nom;
EXCEPTION
  WHERE NO_DATA_FOUND THEN  nom:= 'INEXISTENTE';
  RETURN nom;
END
```

2.10. Ejecución de procedimientos

La interfaz **CallableStatement** permite que se pueda llamar desde Java a los procedimientos almacenados. Para crear un objeto se llama al método *prepareCall(String)* delobjeto *Connection*.

Ejemplo. Declaración de la llamada al procedimiento *subida_sal* que tiene dos parámetros y se les da valor con el marcador de posición **?**.

```
String sql = "{call subida_sal(?,?)}";
CallableStatement llamada=conexion.prepareCall(sql);
```

Cuatro formas de declarar llamadas a los procedimientos y funciones dependiendo de si hay o no parámetros y de la devolución de valores.

- ✓ *[call procedimiento}* para un procedimiento almacenado sin parámetros.
- ✓ *{?=call function}* para una función almacenada que devuelve un valor y no recibe parámetros. El valor se recibe a la izquierda del igual y es el primer parámetro.
- ✓ *{call procedimiento(?, ?, ...)}* para un procedimiento almacenado que recibe parámetros.
- ✓ *{call function(?, ?, ...)}* para una función almacenada que devuelve un valor (primer parámetro) y recibe varios parámetros.

2.11. Gestión de errores

Hasta ahora, cuando se producía un error la secuencia de llamadas al método que ha producido el la excepción y la línea de código donde se producía el error se visualizaba con *printStackTrace()*.

Cuando se produce un error con **SQLException** podemos acceder a cierta información usando los siguientes métodos.

- *getMessage().* Devuelve una cadena que describe el error.
- *getSQLState().* Es una cadena que contiene un estado definido por el estándar X/OPEN SQL.
- *getErrorCode().* Es un entero que proporciona el código de error del fabricante.

```
try
  { // cídigo }
catch (ClassNotFoundException cn) {cn.printStackTrace));}
catch {SQLException e)
{
  System.out.println("HA OCURRIDO UNA EXCEPCIÓN:");
  System.out.println("Mensaje   :"+e.getMessage());
  System.out.println("SQL estado:"+e.getSQLState());
  System.out.println("Cod error :"+e.getErrorCode());
}
```

2.12. Patrón modelo-vista-controlador. Acceso a datos

Se trata de un patrón de diseño utilizado como guía en el diseño de arquitecturas software que ofrecen una fuerte interactividad con el usuario. Para ello se organiza la aplicación en 3 bloques, especializado cada uno en una tarea:

- **El modelo.** Representa los datos de la aplicación y sus reglas de negocio.
- **La vista.** Es la representación del modelo de forma gráfica para interactuar con el usuario; por ejemplo los formularios de E/S o páginas HTML.
- **El controlador.** Interpreta los datos que recibe el usuario para que la aplicación produzca los resultados esperados.

Las ventajas de hacer uso de este patrón son:

- ✓ Separación de los datos de la representación visual de los mismos.
- ✓ Diseño de aplicaciones modulares.
- ✓ Reutilización de código.
- ✓ Facilidad para probar las unidades por separado.
- ✓ Facilita el mantenimiento y la detección de errores

2.12. Patrón modelo-vista-controlador. Acceso a datos

El flujo general de la solicitud de un usuario sería el siguiente:

- El usuario realiza una solicitud a través de una aplicación (pulsar un botón, un enlace...). La solicitud es dirigida al controlador.
- El controlador examina la solicitud y decide qué regla de negocio (modelo) aplicar.
- El modelo contiene las reglas de negocio que procesan la solicitud y que dan lugar al acceso a los datos que necesita el usuario. Esos datos se devuelven al controlador.
- El controlador toma los datos que devuelve el modelo y selecciona la vista en la que se van a presentar esos datos al usuario.
- El controlador devuelve los resultados al usuario tras procesar la solicitud.

Los frameworks que utilizan este patrón son varios: JEE, Apache, etc.

La mayoría de los frameworks para web implementan este patrón.

2.12. Patrón modelo-vista-controlador. Acceso a datos

Una aplicación de este patrón en aplicaciones web J2EE es lo que se conoce como **Modelo 2** y se basa en los siguientes elementos:

- La utilización de Servlets para capturar las peticiones realizadas por el cliente web y redirigirlas a las páginas JSP adecuadas, actuando como controlador.
- Páginas JSP utilizadas para mostrar la interfaz de usuario implementan la vista. Estas páginas usan los JavaBeans como componente modelo.
- JavaBean o POJOs (*Plan Old Java Object*) que implementa el modelo.

Para poder trabajar con la arquitectura Modelo 2 se necesita instalar un contenedor web con un soporte para Servlets y JSPs, como por ejemplo Tomcat.

Los Servlets son clases Java que no tienen el método *main()*; en su lugar se invocan otros métodos cuando se reciben las peticiones: *init(), service() y destroy()*

2.12. Patrón modelo-vista-controlador. Acceso a datos

Los pasos que conforman el ciclo de vida de un Servlet son:

- El cliente solicita una petición a un servidor a través de una URL.
- Si es la primera vez, el servidor carga el Serlet y se llama al método *init()* para iniciarlo.
- Se llama al método *service()* para procesar las peticiones de los clientes web.
- Se llama al método *destroy()* para eliminar al Servlet y liberar los recursos.

Las páginas JSP permiten generar contenido dinámico en la web; son páginas web con etiquetas especiales y código Java incrustado.

Una página JSP consta de dos partes:

- ✓ HTML o XML para el contenido estático.
- ✓ Etiquetas JSP y scriplets escritos en lenguaje de programación Java para encapsular la lógica que genera el contenido dinámico.

2.12. Patrón modelo-vista-controlador. Acceso a datos

Los pasos que conforman el ciclo de vida de un Servlet son:

- El cliente solicita una petición a un servidor a través de una URL.
- Si es la primera vez, el servidor carga el Serlet y se llama al método *init()* para iniciarlo.
- Se llama al método *service()* para procesar las peticiones de los clientes web.
- Se llama al método *destroy()* para eliminar al Servlet y liberar los recursos.

Las páginas JSP permiten generar contenido dinámico en la web; son páginas web con etiquetas especiales y código Java incrustado.

Una página JSP consta de dos partes:

- ✓ HTML o XML para el contenido estático.
- ✓ Etiquetas JSP y scriplets escritos en lenguaje de programación Java para encapsular la lógica que genera el contenido dinámico.

El código de las partes dinámicas se encierra entre etiquetas especiales, la mayoría de las cuales empiezan con <% y terminan con %>. (pág. 101)

2.12. Patrón modelo-vista-controlador. Acceso a datos

Dentro de la carpeta webapps hay algunos ficheros y carpetas que conviene destacar.

Carpeta **WEB-INF.** Es imprescindible. En mayúsculas y contiene:

- Carpeta **classes**. Necesaria si usamos ficheros *.class.* Contiene los paquetes de nuestras clases Java. En este directorio residen los Servlets.
- Carpeta **lib.** Contiene todos los JAR que necesigta la aplicación, como el conector para acceder a una base de datos MySQL.
- Fichero **web.xml**. Indica la ubicación de los servlets (mapeo) contenidos en la aplicación.